

Case Statement

- Helps us to create flags in our data
- Helps us to build logical conditions
- A case statement is normally created within the select statement
- A case statement creates new column

Select

Case

when condition 1 Then result 1

Else default-result

End AS new-column-name

From table;

→ You will get the default-result when all the outlined conditions are

Sales		Price
sale_id	Store	amount
1	Pep	100
2	Pep	50
3	Mr price	80
4	Mr price	20

0-40 Low price
41-60 med price
61->60 High price

SELECT

Case

When price BETWEEN 0 AND 40 Then 'Low Price'

When price BETWEEN 41 AND 60 THEN 'Med Price'

Else 'High Price'

END AS price_group

From sales;

sales_id	store	Price	Price_group
1	Pep	100	High Price
2	Pep	50	Med Price
3	Mr price	80	High Price
4	Mr Price	20	Low Price

Select

Case

When price BETWEEN 0 AND 99 Then 'Budget'

When price BETWEEN 100 AND 1000 Then 'med-range'

Else 'Expensive'

End AS Price-category

From Products;

- A case statement help us to classify, to group, to bucket our data
- AS case statement help us to create a categorical column from a numeric column based on specific conditions

```
SELECT product_id,  
       product_name,  
       Price
```

Case

```
       when price > 100 Then 'Expensive'  
       when price Between 100 AND 100 Then 'Mid-range'  
       when price < 100 Then 'Budget'  
AND AS 'price category'  
FROM Products;
```

SELECT - select specific columns from table
FROM - to specify the table name we selecting data from
database.schema.table

AS - give a column a descriptive name
- to rename a column

Where - filtering

- filter rows
- filter specific rows from a table

Comparison Operator

=	Equal to
!= or <>	Not Equal to
>> >	Greater than
# >=	Greater Greater than or Equal to
<= <	Less than
<=	Less than or Equal to

Logical Operators - Help us to build specific Logic

AND - Both conditions must be met

OR - one condition must be met

IN - specify values

Aggregate Functions

- Summarize data
- Consolidate data
- combines data into groups
- Performs calculation on multiple rows & return single value per group

- 1) MIN()
- 2) MAX()
- 3) Count()
- 4) AVG()
- 5) SUM()

- The aggregate functions are used with the group by statement
- Having statement is used to filter aggregate data

ORDER BY

- Sorting statement used to arrange the data either in ascending order or descending order

ASC - Ascending

- small to big

DESC - Descending order

- Big to small

Case Statement

- Helps us to create flags in our data
- build logical conditions
- Create a new column
- Placed within the Select statement
- Creates logic
- Creates a categorical column from a numeric column.

Case

when condition 1 THEN 'results'

when condition 2 THEN 'results2'

Else 'default-result'

END AS new-column-name

→ Given when condition 1 & condition

Count (Distinct ID)

- used to count the number of different values in specific column

column name
ID
101
102
101
103

Select (count distinct id)
from employees;

3

1) write a sql query to find the total number of employees in the company

Select Count(ID) AS number-of-employees
from employees;

2) Write a sql query paid to all employees in the IT department

Select department
Sum(salary) AS total-salary
from employees
where department = 'IT';

4) MIN & MAX

write a sql query to find the highest & lowest salary in the company

Select Min(salary) AS lowest-salary
from employees;

Select Max(salary) AS Highest-salary
from employees;

5) Write a sql query to group employees by department & display the total salary paid in each department

● Select department
SUM(salary) AS total-salary
from employees
Group by department

Case statement

- build conditional logic
- Creates a new column
- Placed within the select statement

● Select

case

when condition Then 'result1'

when condition2 Then 'result2'

else 'default result'

End AS 'column-name'

from table

Select customer-name
amount;

Case

When amount > 1000 Then 'High value'

When amount BETWEEN 500 AND 999.99 THEN 'Medium'

When amount < 500 Then 'Low value'

End AS order-value-category
from orders;

NULL is an empty space (no data)
- missing value

Select emp-name,
department,
salary;

Case

When amount > 80000 Then 'Senior II'

When amount > 55000 Then 'Experienced'

When Else 'staff'

End AS position-level
from employees;

Select emp-name,
department,
salary;

Case

When department = 'IT' AND salary > 80000 Then

When department = 'HR' AND salary > 55000 Then

Else 'staff'

End AS position-level
from employees;